

A **web server** is a core system that powers the entire web ecosystem. It serves as the bridge between users and web applications, handling requests and delivering responses over the Internet. The term “web server” can refer to both the hardware that stores website data and the software that processes user requests using protocols like HTTP and HTTPS. Whenever a user enters a URL, such as *www.example.com*, their browser sends a request to the web server. The server retrieves the requested files (like HTML, CSS, and JavaScript) and sends them back to the client’s browser, displaying the website. This process happens in milliseconds and forms the foundation of all web communication.

Web Server Architecture

The architecture of a web server determines how efficiently it handles multiple client requests and responses. Modern web servers follow a **modular and layered design**, ensuring high performance and flexibility.

At the heart of every web server lies the **core engine**, which manages connections, interprets requests, and coordinates responses. Surrounding this core are multiple **modules**—each responsible for specific functions such as SSL encryption, URL rewriting, or caching.

The process of handling a request typically includes: 1. **Request Reception:** The server receives an HTTP request from the client. 2. **Processing:** It interprets the method (GET, POST, PUT, DELETE) and checks for the requested resource. 3. **Execution:** If it’s a static file, it’s sent directly; if it’s dynamic, the server interacts with scripts (like PHP or Node.js) to generate a response. 4. **Response Delivery:** The response, along with headers and status codes, is sent back to the client. 5. **Logging:** The server logs all requests for monitoring and analytics.

Example: When you visit *www.puacp.edu.pk*, the web server processes your request, fetches `index.html`, and sends it to your browser while logging your visit.

Virtual Hosting

Virtual hosting allows a single physical server to host multiple websites, enabling efficient use of resources. Instead of requiring separate servers for each domain, web administrators can configure virtual hosts to serve many sites from one machine.

There are three main types of virtual hosting:

1. **Name-Based Virtual Hosting:** This is the most common and cost-effective type. It uses a single IP address for multiple domains. The server identifies which site to serve based on the *Host* header in the HTTP request.

2. **Example:** `example.com` and `noorayy.com` both hosted on 192.168.1.10.
3. **Advantages:** Economical, easy to configure.
4. **Disadvantages:** SSL certificate configuration can be complex without SNI.
5. **IP-Based Virtual Hosting:** Each site is assigned its own IP address. The server uses these unique IPs to distinguish between websites.
6. **Example:** `portal.puacp.edu.pk` and `library.puacp.edu.pk` each have different IPs.
7. **Advantages:** Better security and SSL management.
8. **Disadvantages:** Requires more IP addresses, increasing costs.
9. **Port-Based Virtual Hosting:** Different websites share the same IP but use separate port numbers (e.g., port 80 and port 8080).
10. **Example:** Internal testing websites using port variations.
11. **Advantages:** Simple for private or local network configurations.
12. **Disadvantages:** Not user-friendly for public domains.

Virtual hosting makes it possible for a single web server to handle hundreds of websites, as seen in shared hosting environments. For instance, **PUACP** could host `portal.puacp.edu.pk`, `students.puacp.edu.pk`, and `faculty.puacp.edu.pk` all on one physical server.

Chunked Transfer Encoding

Chunked Transfer Encoding is a mechanism of **HTTP/1.1** that allows a web server to send data in parts or chunks rather than waiting for the complete data to be available. This is essential for dynamically generated content or streaming services.

When a server cannot determine the content length beforehand—like in a live news feed—it uses chunked transfer encoding. Each chunk is sent with its size (in hexadecimal), followed by the chunk data and a carriage return + line feed (CRLF). Once all data has been sent, a zero-length chunk marks the end.

Example of Chunked Encoding Format:

```
4\r\nWiki\r\n
5\r\npedia\r\n
0\r\n\r\n
```

This data represents the word “Wikipedia” in two chunks.

Encoding Process: 1. The server divides data into manageable chunks. 2. Each chunk begins with its size header (in hexadecimal). 3. The chunked message ends when the size is `0`.

Decoding Process: 1. The client reads the chunk size header. 2. It then reads the chunk data accordingly. 3. All chunks are combined to recreate the complete response.

Advantages: - Reduces latency for large or dynamic data. - Allows browsers to render pages progressively. - Essential for real-time streaming, chat apps, and live updates.

Web Caching

Web caching is the process of storing frequently accessed web content in temporary storage for faster access in future requests. It ensures that users don't have to download the same resources repeatedly, improving performance and reducing server load.

How Web Caching Works

When a user visits a website, the browser sends a request to the server. The server processes the request, generates a response, and sends it back. Caching systems—whether in the browser, proxy, or server—intercept and store this response. On subsequent requests, the cached version is served instead of generating a new one, saving both time and bandwidth.

Why Every Web Service Needs a Caching Layer

Without caching, every request reaches the origin server. This puts excessive pressure on the system, leading to slow response times. Caching acts as a buffer that minimizes server load, reduces network congestion, and enhances the user experience. It's essential for high-traffic websites like Google, YouTube, or Facebook, where millions of users request similar content simultaneously.

Advantages and Benefits of HTTP Caching

- Reduces bandwidth usage.
- Improves page load speed and user experience.
- Lowers the operational cost of servers.
- Enhances scalability and network performance.

Types of Web Caching

1. **Browser Cache:** A client-side cache stored in the user's browser. When visiting a webpage, the browser saves static files (images, CSS, JavaScript). On revisiting, the browser loads these from local storage instead of downloading them again, saving bandwidth and time.
2. **Proxy Cache:** A server-side cache managed through a proxy server positioned between the client and the origin server. If a requested resource exists in the cache, it's delivered directly to the client. Proxy caches can serve multiple users simultaneously, improving performance for all clients.
3. **Server Cache:** Stored on the origin server. Frequently requested pages or database queries are cached to avoid redundant computations.

These caching systems together ensure smoother browsing and faster content delivery.

Extensibility of Web Servers

Extensibility in web technology allows developers to enhance and customize web server functionalities using different mechanisms such as plugins, APIs, and templates.

Types of Extensibility in Web Technology

1. **Plugin-Based Extensibility:** Developers create custom plugins or modules to extend functionality. Example: WordPress plugins or browser extensions.
2. **API-Based Extensibility:** APIs allow external systems to interact with the web application, adding features or integrating services. Example: REST APIs, GraphQL.
3. **Component-Based Extensibility:** Breaks applications into reusable modules, making it easier to update or modify specific features without affecting the entire system.
4. **Event-Based Extensibility:** Developers define event handlers that trigger specific actions within the system (e.g., sending notifications on new user registration).
5. **Template-Based Extensibility:** Allows customization of design and layout using templates or themes. Example: Pre-built WordPress or Shopify themes.

These extensibility types make web applications more flexible, modular, and scalable. They enable developers to add new features without altering the core system code.

Server Configuration

Configuration files define how web servers operate. They specify document roots, ports, and access permissions. Proper configuration ensures performance, security, and reliability.

Common directives include: - **DocumentRoot:** Directory containing site files. - **ServerName:** Defines domain name. - **Listen:** Port specification (80 for HTTP, 443 for HTTPS). - **Directory Controls:** Manage file permissions. - **ErrorLog/AccessLog:** Monitor errors and access records.

Server Security

Web server security is crucial because servers are public-facing systems vulnerable to attacks like DDoS, SQL injection, and XSS. Security measures include: - Enabling HTTPS with SSL/TLS. - Regular updates and patches. - Firewalls and intrusion detection. - Strong password and permission management. - Disabling directory listings.

Example: Enabling HTTPS ensures secure data transmission, protecting sensitive user information from interception.

End of Topic 3 – Web Servers (Final Expanded Version)